哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

高级语言程序设计
High-level Language Programming

Lecture 12: Files and Streams

Files and Streams

Course Overview

- Input/output class hierarchy
- Basic I/O operations
- Object and Binary I/O

Try it yourself on https://www.onlinegdb.com

12.1 Input/output class hierarchy

- The input-output statements used so far were those that read data from the keyboard and displayed data on the screen.
  - A program stores the data from the keyboard in the computer's memory.
  - When the program terminates, the data is lost and must be reentered every time the program is run.
- File stream input and output
  - Files use external storage devices, such as hard disks and USB keys to store data.
  - These are permanent storage devices that allow data to be stored after the program terminates.

12.1 Input/output class hierarchy

- C++ has no built-in input/output (I/O) commands. In C++ the I/O commands are included in a class library.

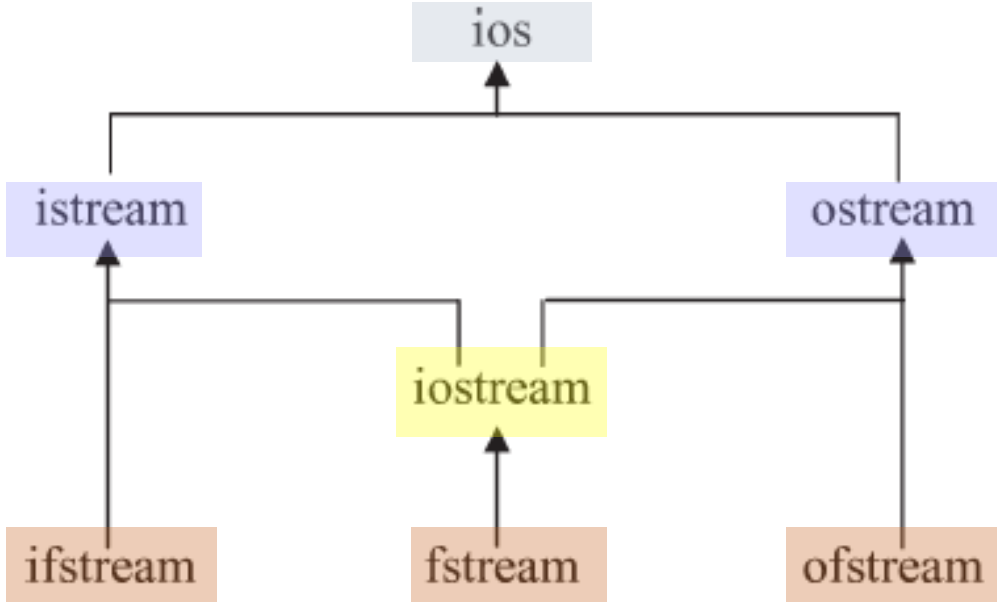


Figure 14.1 I-O Class hierarchy

The base class ios contains data members to indicate conditions such as whether a stream object is opened for input or output and whether the end of the file has been reached.

12.1 Input/output class hierarchy

- The derived class *istream* extends the base class by adding member functions to input data from a stream.
  - This class provides basic input processing by including overloaded operators >> for reading the built-in data types (char, int, float etc.) from a stream.

- The derived class *ostream* contains member functions to output data to a stream.
  - This class provides basic output processing by including overloaded insertion operators (<<) for writing built-in data types to a stream.
  - The stream ***cout*** is an object of this class and is normally attached to the screen.

12.1 Input/output class hierarchy

- The class *ifstream* is derived from istream and is used to create input file objects.
- The class *ofstream* is derived from ostream and is used to create output file objects.
- The class *fstream* is used to create file objects that can be used for both input and output.
- The definition of the library classes is contained in the header files **iostream** and **fstream**

`iostream`	Includes the definition of • `ios`, `istream`, `ostream`, and `iostream` classes • stream objects `cin`, `cout`, `clog` and `cerr` • stream manipulators `endl`, `ws`, `dec`, `hex` and `oct`.
`fstream`	• Includes the definition of the `ifstream`, `ofstream`, and `fstream` classes.

12.2 Basic I/O operations

- Opening a file
  - first create an instance of the appropriate class

```
ifstrem in ;      // in is an input file object
ofstream out ;    // out is an output file object
```

  - After creating an instance of the appropriate class, the file object must be opened to associate it with a file stored on a hard disk or some other storage device.

```
in.open( "in.dat" ) ;     // in is associated with in.dat
out.open( "out.dat" ) ;   // out is associated with out.dat
```

  - Creating an **instance of a file object** and opening a file can be combined into one statement by **using** the *ifstream* and *ofstream* **class constructors**.

```
ifstrem in( "in.dat" ) ;
ofstream out( "out.dat" ) ;
```

12.2 Basic I/O operations

```
3  #include <iostream>
4  #include <fstream>
5  #include <string>
6  using namespace std ;
8  main()
9  {
10   char c = 'A' ;
11   int i = 1 ;
12   string s = "Hello" ;
14   ofstream out ;                  // Create a file object out and
15   out.open( "file.txt" ) ;        // open file.txt
17   // Write some data to the file using <<
18   out << s << ' ' << c << ' ' << i << endl ;
19   out.close() ;  // Close the output file.
21   ifstream in( "file.txt" ) ;  // Use constructor to open the file.
22   // Read data from the file using >>
23   in >> s >> c >> i ;
24   in.close() ;  // Close the input file.
26   cout << "Data read from file:" ;
27   cout << s << ' ' << c << ' ' << i << endl ;
28 }
```

- Line 14 creates an instance out of an output file object.
- Line 15 associates out with the external file file.txt and opens the file for processing. If the file does not exist, by default it will be created in the same folder (directory) as the program.
- Line 18 outputs some data to the file stream using << in the same way that data would be sent to the stream cout.

- The ifstream constructor is used on line 21 to open file.txt for input processing.
- Line 23 is similar to the way data is input from the keyboard, except in instead of cin.

12.2 Basic I/O operations

- **File error checking: possible errors**
  - No space being available on an output device
  - Attempting to open a non-existent file for reading.
- The *ios* class contains member functions that can be used to check for errors.
  - Since ifstream and ofstream are inherited from ios, these functions are available for use with ifstream and ofstream objects.

It is good practice to check for errors when opening a file!

12.2 Basic I/O operations

Member Function	Return Value
`fail()`	Returns the Boolean value `true` if the operation failed; otherwise the Boolean value `false` is returned.
`good()`	Returns the Boolean value `true` if the operation succeeded; otherwise the Boolean value `false` is returned. This is the opposite of `fail()`.
`eof()`	Returns the Boolean value `true` if the end of the file has been reached; otherwise the Boolean value `false` is returned.

- The member function fail() to check for errors when opening a file.
  - A return value of zero indicates success, while a non-zero return value indicates an error.

12.2 Basic I/O operations

```
3   #include <iostream>
4   #include <fstream>
5   #include <string>
6   using namespace std ;
7
8   main()
9   {
10    char c = 'A' ;
11    int i = 1 ;
12    string s = "Hello" ;
13    ofstream out ;
14
15    out.open( "file.txt" ) ;
16    if ( out.fail() )  // Has the file failed to open?
17    {
18      cerr << "Failure to open file.txt for output" << endl ;
19      return 1 ;
20    }
```

- The additional program statements can be tested by adding a non-existent storage device identifier to the file path, e.g. instead of "file.txt" specify "xx:/file.txt" on lines 15 and 25.

12.2 Basic I/O operations

```
21    // Write some data to the file using <<
22    out << s << ' ' << c << ' ' << i << endl ;
23    out.close() ;
24
25    ifstream in( "file.txt" ) ;
26    if ( in.fail() )
27    {
28      cerr << "Failure to open file.txt for input" << endl ;
29      return 1 ;
30    }
31    // Read data from the file using >>
32    in >> s >> c >> i ;
33    in.close() ;  // Close the input file.
34    cout << "Data read from file:" ;
35    cout << s << ' ' << c << ' ' << i << endl ;
36    return 0 ;
37 }
```

12.2 Basic I/O operations

- When reading data from an input file, it is necessary to be able to detect the end of the file so that processing of the file may stop.
  - **The end of a file may be detected** using the *ios* member function ***eof()*** inherited by the *ifstream* class.
- The following program example
  - demonstrates the use of *eof()* in making a copy of a text file.
  - The *istream* member function *get()* is used to **read a character** from the input file
  - The *ofstream* member function *put()* is used to **write the character** to an output file.

12.2 Basic I/O operations

```
2  // Program to demonstrate member functions eof, get and put.
3  #include <iostream>
4  #include <fstream>
5  using namespace std ;
6
7  main()
8  {
9    char c ;
10   int count ;
11
12   ifstream in( "file.txt" ) ;  // Open the input file.
13   if ( in.fail() )
14   {
15     cerr << "Open failure on file.txt" << endl ;
16     return 1 ;
17   }
18
19   ofstream out( "file.bak" ) ;  // Open output file.
```

12.2 Basic I/O operations

```
20    if ( out.fail() )
21    {
22      cerr << "Open failure on file.bak" << endl ;
23      return 1 ;
24    }
25
26    in.get( c ) ; // Read the first character in the file.
27    count = 1 ;
28    while( !in.eof() ) // Loop while not end of file.
29    {
30      out.put( c ) ;  // Write the character.
31      in.get( c ) ;   // Read the next character.
32      count++ ;
33    }
34
35    in.close() ;
36    out.close() ;
37
38    cout << "Copy completed. "
39         << count - 1 << " characters copied." << endl ;
40    return 0 ;
41  }
```

- Line 26 reads the first character of the input file.
- Lines 28 to 33 continues to write a character to the output file and read the next character from the input file until the end of the input file is detected.
- The program can be improved by using an easier while statement than that used on line 28:

```
while ( in.get( c ) )
```

12.2 Basic I/O operations

```
4   #include <fstream>
5   #include <iomanip>
6   #include <string>
7   using namespace std ;
8
9   main()
10  {
11    string word ;
12    int word_count = 0 ;
```

```
14    ifstream in( "words.txt" ) ;
15    if ( in.fail() )
16    {
17      cerr << "Open failure on words.txt" << endl ;
18      return 1 ;
19  }
20
21    while ( in >> word ) // Read a word while not end of file.
22    {
23      cout << word << endl ;
24      word_count++ ;
25    }
26
27    in.close() ;
28    cout << "Number of words read = " << word_count << endl ;
29    return 0 ;
30  }
```

- Detecting the end of file can be applied to any member function that returns a reference to a stream using the stream extraction operator >>
- The program assumes that each "word" is delimited by one or more whitespace characters. Whitespace characters are ignored by >>.

•The while loop from line 21 to line 25 continues to display each "word" in the file until the end of the file is detected.
•When the end of the file occurs, the while loop terminates, the file is closed.

12.2 Basic I/O operations

- To append data to the end of a file, the file mode must be specified when opening the file.
- The file mode values are specified in the **ios** class.

Mode	Meaning
`ios::app`	Opens a file for appending - additional data is written at the end of the file.
`ios::ate`	Start at the end of the file when a file is opened.
`ios::binary`	Opens a file in binary mode (default is text mode).
`ios::in`	Opens a file for input (default for `ifstream` objects).
`ios::out`	Opens a file for output (default for `ofstream` objects).
`ios::trunc`	Truncates (deletes) the file contents if the file already exists.
`ios::nocreate`	Do not create a new file. Open fails if the file does not already exist. For output files only.
`ios::noreplace`	Do not replace an existing file. Open fails if the file already exists. For output files only.

12.2 Basic I/O operations

- These values may be combined using the bitwise operator OR (|).
  - open a file for input and output

```
fstream in_out( "file.txt", ios::in | ios::out ) ;
```

  - the default open mode:
    - for *ifstream* objects is **ios:: in**
    - for *ofstream* objects is **ios::out**

12.2 Basic I/O operations

Program Example : demonstrates appending the line “This line is added to the end of the file” to the file file.txt.

```
3  #include <iostream>
4  #include <fstream>
5  using namespace std ;
6
7  main()
8  {
9    ofstream out( "file.txt", ios::app ) ;
10   if ( out.fail() )
11   {
12     cerr << "Open failure on file.txt" << endl ;
13     return 1 ;
14   }
15   out << "This line is added to the end of the file" << endl ;
16   out.close() ;
17   return 0 ;
18 }
```

12.2 Basic I/O operations

- *getline()*
  - An entire line can be read from a file into either a C-string or a C++string using the function *getline()*.

```
2  // Program to demonstrate reading a file line by line
3  // using a C++ style string.
4  #include <iostream>
5  #include <fstream>
6  using namespace std ;
7
8  main()
9  {
10   string cpp_string ; // A C++ style string.
11   int line_number = 0 ;
```

```
13   ifstream in( "p14g.cpp" ) ; // Open this program file.
14   if ( in.fail() )
15   {
16     cerr << "Open failure on p14g.cpp" << endl ;
17     return 1 ;
18   }
19   // Read each line into a C++ string,
20   // and display the line number and the string.
21   while ( getline( in, cpp_string) )
22   {
23     cout << ++line_number << ":" << cpp_string << endl ;
24   }
25   in.close() ;
26   return 0 ;
27 }
```

12.2 Basic I/O operations

- Program Example: uses a C string for the same purpose

```
3  // using a C-style string.
4  #include <iostream>
5  #include <fstream>
6  using namespace std ;
7
8  main()
9  {
10   char c_string[81] ; // A C-style string, maximum size is 80.
11   int line_number = 0 ;
12
13   ifstream in( "p14h.cpp" ) ;
14   if ( in.fail() )
15   {
16     cerr << "Open failure on p14h.cpp" << endl ;
17     return 1 ;
18    }
19   // Read each line into a C-string
20   // and display the line number and the string.
21   while( in.getline ( c_string, 81 ) )
22   {
23     cout << ++line_number << ':' << c_string << endl ;
24   }
25   in.close() ;
26   return 0 ;
27 }
```

12.3 Object and Binary I/O

- Object I/O
  - We can exploit the >> and << operators, overloading them for keyboard input and screen output operations specific to the time24 class.
  - No modifications to the time24 class are needed to use disk files instead of the keyboard and screen.

12.3 Object and Binary I/O

```
2   // Demonstration of object I/O.
3   #include <iostream>
4   #include <fstream>
5   #include "time24.h"
6   #include "time24.cpp"
7   using namespace std ;
8
9   main()
10  {
11    time24 t1( 1, 2, 3 ) ;
12    time24 t2( 10, 10, 10 ) ;
13
14    fstream in_out( "times.dat", ios::in|ios::out ) ;
15    if ( in_out.fail() )
16    {
17      cerr << "error on opening times.dat" << endl ;
18      return 1 ;
19    }
```

Both input and output.

- The file time24.h included on line 5 contains the time24 class declaration
- Line 6 includes the file time24.cpp containing the time24 class member functions
- Line 14 opens the file times.dat as an *fstream* object for input and output.

12.3 Object and Binary I/O

```
21    // Write objects to the file.
22    in_out <<  t1 << t2 ;
23
24    // Rewind the file to the start.
25    in_out.seekg( 0 ) ;
26
27    // Read objects back and display on the screen.
28    in_out  >> t1 >> t2 ;
29    cout << t1 << t2 ;
30    in_out.close() ;
31    return 0 ;
32 }
```

- Line 22 writes the time24 objects t1 and t2 to the file.
- Line 28 reads the objects back from the file and line 29 displays the objects.

12.3 Object and Binary I/O

- Types of files in C++
  - *text (or ASCII) files*
    - numeric data is stored as ASCII characters
  - *binary* files.
    - numeric data is stored in binary format

```
short int n  = 123 ;
```

Example

the variable n occupies 2 bytes of memory.
storing the value of n in a text file requires 3 bytes of memory

Character :	'1'	'2'	'3'
ASCII value in decimal:	49	50	51
ASCII value in binary:	00110001	00110010	00110011

12.3 Object and Binary I/O

- ASCII file
  - Each digit of the number requires one byte of storage.
- binary file
  - The digits of a number do not occupy individual storage locations. Instead the number is stored in its entirely as a binary number.
  - Example:
    - n=123
    - n=1234: the same storage

00000000	01111011
0000100	10010010

An ASCII file would require more bytes to store the extra digit.

12.3 Object and Binary I/O

- **write()**
  - The two-argument *istream* member function
  - Be used to write an object in binary format.
  - The first argument is the address of the block of memory where the object is stored and the second argument is the size of the block in bytes.
- The object must be stored in one contiguous block of memory
  - The object cannot have a pointer data member to dynamically allocated memory.
  - The class cannot have any *string* data members, since the *string* class uses a pointer data member.

12.3 Object and Binary I/O

- The file *stock.h*

```
4   // Header for a simple stock class.
5   #include <fstream>
6   using namespace std ;
7
8   class stock
9   {
10  public:
11    stock() ;
12    // Purpose: Default constructor.
13
14    stock( int code, char desc[ ], int quantity ) ;
15    // Purpose    : Constructor
16    // Parameters: stock code, description and stock quantity.
17    //           : Note that the description is a C-string.
```

The file *stock.cpp*

```
4   // Member function definitions for stock class.
5   #include "stock.h"
6   #include <cstring>
7   stock::stock()
8   {
9     stock_code = 0 ;
10    quantity_in_stock = 0 ;
11    description[0] = '\0' ;
12  }
13
14  stock::stock( int code, char desc[ ], int quantity )
15  {
16    stock_code = code ;
17    strcpy( description, desc ) ;
18    quantity_in_stock = quantity ;
19  }
```

```
42  private:
43    int stock_code ;
44    char description[21] ;
45    int quantity_in_stock ;
```

12.3 Object and Binary I/O

- The file *stock.h*

```
23   const int get_code () const ;
24   // Purpose: Inspector function to return the stock code.
25
26   const char* get_description() const ;
27   // Purpose: Inspector function to return the stock description.
28   //        : Note that description is a C-string.
29
30   const int get_quantity () const ;
31   // Purpose: Inspector function to return the stock quantity.
```

The file *stock.cpp*

```
26 const int stock::get_code () const
27 {
28   return stock_code ;
29 }
30
31 const char* stock::get_description() const
32 {
33   return description ;
34 }
35
36 const int stock::get_quantity() const
37 {
38   return quantity_in_stock ;
39 }
```

12.3 Object and Binary I/O

```
2  // Demonstration of writing to a binary file.
3  #include <iostream>
4  #include "stock.h"
5  #include "stock.cpp"
6  using namespace std ;
7
8  main()
9  {
10   // Initialise an array of stock objects.
11   class stock stationery[ 10 ] =
12   { stock( 1, "Pencil", 68 ),
13     stock( 2, "Pen", 30 ),
14     stock( 3, "Highlighter", 90 ),
15     stock( 4, "Eraser", 24 ),
16     stock( 5, "Pencil Sharpener", 5 ),
17     stock( 6, "Pocket Folder", 50 ),
18     stock( 7, "Paper Tie", 300 ),
19     stock( 8, "Glue Stick", 30 ),
20     stock( 9, "Box File", 35 ),
21     stock( 10, "Note Book", 97 ) } ;
```

```
23   int object_size = sizeof( stock ) ;
24
25   // Open the stock file for binary output.
26   ofstream stock_file( "stock.dat", ios::binary ) ;
27   if ( stock_file.fail() )
28   {
29     cerr << "Open failure on stock.dat" << endl ;
30     return 1 ;
31   }
```

Line 23 calculates the second argument in *write()*, the number of bytes in the memory block

12.3 Object and Binary I/O

```
33    // Write each object to the output file.
34    for ( int i = 0 ; i < 10 ; i++ )
35    {
36      cout << "Writing Stock Code "
37           << stationery[ i ].get_code() << ' '
38           << stationery[ i ].get_description()
39           << " quantity = " << stationery[ i ].get_quantity()
40           << endl ;
41      stock_file.write( reinterpret_cast<char *>( &stationery[i] ),
42                        object_size ) ;
43    }
44    stock_file.close() ;
45    return 0 ;
46 }
```

- The write() function on line 41 treats the stock object as a block of memory bytes or characters, which it copies from the memory to a disk file without converting to ASCII.

12.3 Object and Binary I/O

- The first argument in write() is a pointer to a character string.
  - The object is treated as a block of characters, so the address of the *ith* stock object, *&stationery[i],* is cast to a pointer to a character string by

```
reinterpret_cast<char*>( &stationery[i] )
```

  - The **reinterpret_cast** allows any pointer type to be converted to any other pointer type.
- The second argument in write() is the number of bytes in the memory block

12.3 Object and Binary I/O

- **Serial reading of objects from a binary file read()**
  - The *istream* member function
  - Be used to read an object's data from a binary file into memory.
  - This function uses the same arguments as *write()*

```
2  // Demonstration of serial reading of a binary file.
3  #include <iostream>
4  #include <iomanip>
5  #include "stock.h"
6  #include "stock.cpp"
7  using namespace std ;
```

12.3 Object and Binary I/O

```
 9  main()
10 {
11 stock stock_record ;
12
13   // Open the stock file for binary input.
14   ifstream stock_file( "stock.dat", ios::binary ) ;
15   if ( stock_file.fail() )
16   {
17     cerr << "Open failure on stock.dat" << endl ;
18     return 1 ;
19   }
20
21   int object_size = sizeof( stock ) ;
22
23   cout << "Code Description Quantity" << endl ;
24   cout << left ;  // Left justify data.
```

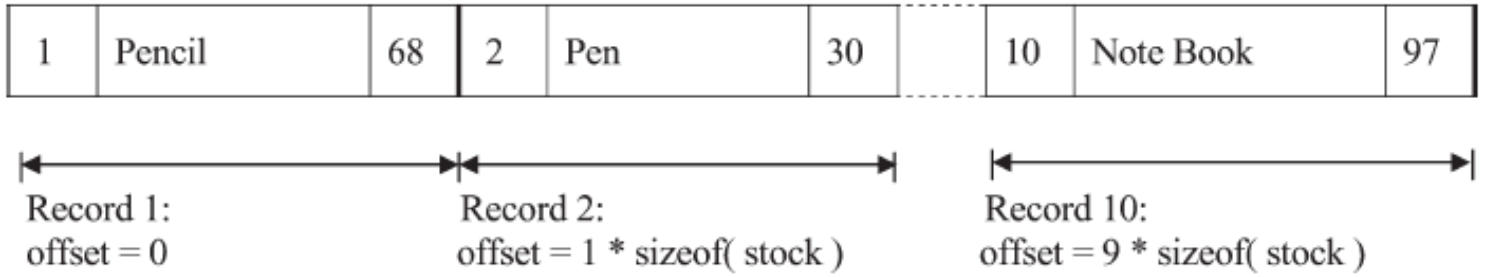
12.3 Object and Binary I/O

```
25    // Read and display stock records until eof.
26    while( stock_file.read
27           ( reinterpret_cast<char *>( &stock_record ),
28             object_size ) )
29    {
30      cout << setw(5) << stock_record.get_code()
31           << setw(20) << stock_record.get_description()
32           << stock_record.get_quantity() << endl ;
33    }
34    stock_file.close() ;
35    return 0 ;
36 }
```

Code	Description	Quantity
1	Pencil	68
2	Pen	30
3	Highlighter	90
4	Eraser	24
5	Pencil Sharpener	5
6	Pocket Folder	50
7	Paper Tie	300
8	Glue Stick	30
9	Box File	35
10	Note Book	97

12.3 Object and Binary I/O

- The layout of the file as created by previous program

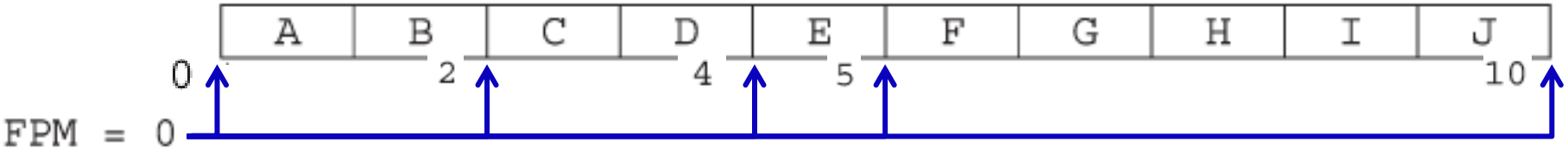


- *Relative file*
  - *The stock* code is related to the position of the record in the file,
    - stock code 1 is the first record,
    - stock code 2 is the second record ,
    - stock code 3 is the third record ,
    - ……

The offset of a stock record in the file can be calculated by subtracting 1 from the stock code and multiplying the result by the size of the stock record.

12.3 Object and Binary I/O

- *seekg()* (meaning 'seek get') is used to set the **file position marker** for an input file, from where the next input data is read.
  - The *istream* member function
  - This function requires two arguments.
    - The first argument is an offset that tells **how many bytes to skip**.
    - The second argument specifies the point from where the offset is measured




```
istream in( "letters.dat" ) ;

in.seekg( 4, ios::beg ) ;
in.seekg( 1, ios::cur ) ;
in.seekg( 0, ios::end ) ;
```

```
in.seekg( 4 ) ;
```